

Compositional Latent Space

Suresh Murugaiyan

July 2026

1 Introduction

We train a *compositional autoencoder* (Path A of the compositional operator-network framework) on the FlowBench 2D lid-driven cavity dataset: steady Navier–Stokes flow with an object placed inside the cavity. The autoencoder compresses each steady flow field into a small latent vector $\mathbf{z}$ and reconstructs the field from it — a nonlinear analogue of POD. The key structural choice is that $\mathbf{z}$ is not one opaque vector; it is partitioned by architecture into three named blocks, each intended to carry one physical factor:

$$\mathbf{z} = [\underbrace{\mathbf{z}_\mu}_{\text{regime (4)}} \parallel \underbrace{\mathbf{z}_g}_{\text{geometry (32)}} \parallel \underbrace{\mathbf{z}_\xi}_{\text{residual (16)}}] \in \mathbb{R}^{52}. \quad (1)$$

The hypothesis under test: after training, $\mathbf{z}_\mu$ should contain the Reynolds number, $\mathbf{z}_g$ should contain the object shape, and neither should leak into the other. Because the flow is steady, there is no dynamics block $\mathbf{z}_\eta$ and no latent propagator Φ in this first version; those enter later with time-dependent cases.

2 Dataset

Each FlowBench sample is a steady lid-driven cavity solution at 512×512 resolution. The data loader (`data/dataset.py`) unpacks the raw `.npz` arrays into:

- **fields** — the (u, v, p) solution, resized $512^2 \rightarrow 256^2$ to keep training cheap;
- **sdf** — the signed distance field of the object (the geometry label);
- **mask** — binary fluid mask (1 in fluid, 0 in the solid);
- **log_re** — the regime label. Re is stored as a constant channel, so a single pixel is extracted, transformed to $\log_{10} \text{Re}$ (Re spans orders of magnitude), and standardized with *train-set* statistics; the test set reuses those statistics so both splits share one scale.

The loader also computes three cheap per-sample geometry summaries from the mask (solid area fraction, centroid x , centroid y). These are never used during training; they serve only as ground-truth targets for the linear-probe diagnostic of Section 5.

3 Network architecture

Four modules (`models/compositional/networks.py`):

- **FieldEncoder** $\mathcal{E}$ — a CNN that halves the spatial resolution repeatedly ($256 \rightarrow 8$), pools, and terminates in *three separate linear heads* producing $\mathbf{z}_\mu, \mathbf{z}_g, \mathbf{z}_\xi$. The blocks are architecturally separate from the start.
- **FieldDecoder** $\mathcal{D}$ — the mirror image: the concatenated $\mathbf{z}$ is reshaped to an 8×8 feature map and upsampled back to a 256^2 (u, v, p) field.
- **RegimeHead** — a small MLP that reads $\mathbf{z}_\mu$ *only* and must predict $\log \text{Re}$.

- **SDFHead** — a small decoder that reads z_g *only* and must redraw the object’s SDF at 64×64 .

The two heads are the supervision mechanism: since the regime head sees only z_μ , the encoder is forced to route Reynolds-number information into that block, and likewise shape information into z_g .

4 Training objective

Each optimization step combines four losses (`models/compositional/compositional_ae.py`), weighted by λ -coefficients set in the configuration file:

1. **Masked reconstruction (L1).** MSE between decoded and true fields, evaluated over fluid points only: predictions inside the solid are zeroed and the error is normalized by the fluid node count, so samples with large objects are not unfairly weighted.
2. **Regime supervision.** MSE of `RegimeHead( $z_\mu$ )` against the standardized $\log_{10}$ Re. Pulls Re into z_μ .
3. **Geometry supervision.** MSE of `SDFHead( $z_g$ )` against the downsampled true SDF. Pulls shape into z_g .
4. **Cross-block decorrelation (L6).** The mean absolute Pearson correlation, computed across the batch, between every coordinate of one block and every coordinate of another, penalized for all three block pairs. Losses 2–3 pull information *into* the correct block; this term pushes redundant copies *out of* the wrong blocks.

The residual block z_ξ receives no supervision at all: it is free capacity that absorbs whatever the reconstruction needs beyond (Re, shape), which is exactly the role of the residual block in the framework.

Training is driven by `main.py`: it loads the YAML configuration, seeds all libraries, builds the train/test datasets (passing the train Re-statistics to the test set), and trains with PyTorch Lightning, checkpointing on the best validation reconstruction error.

5 Linear-probe diagnostics

The diagnostic (`diagnostics/probes.py`) is the actual scientific test. After training, the entire test set is encoded, and for each (block, target) pair a ridge regression is fitted with 5-fold cross-validation; the resulting R^2 values are tabulated. For a compositional latent we expect the block-diagonal pattern of Table 1.

Table 1: Expected linear-probe R^2 pattern for a compositional latent space. High R^2 on the matching block and low R^2 elsewhere indicates a successful factorization; a high off-diagonal entry indicates leakage.

Target	z_μ	z_g	z_ξ
log Re	high	low	low
Area fraction	low	high	low
Centroid x	low	high	low
Centroid y	low	high	low

If the trained latent reproduces this pattern, the factorization worked: Re is linearly readable from the regime block and *not* from the geometry block, and vice versa. A high R^2 in an off-diagonal cell means information leaked — exactly what the planned loss ablations are designed to study (e.g., does removing the decorrelation penalty cause leakage?).

6 Results

6.1 Run 1: baseline loss weights

First training run on the FlowBench LDC “easy” split (2,400 train / 600 test samples): 200 epochs, batch size 16, resolution 256^2 , loss weights $\lambda_{\text{recon}} = 1.0$, $\lambda_{\text{regime}} = \lambda_{\text{geo}} = 0.1$, $\lambda_{\text{decorr}} = 0.01$ (single NVIDIA GPU,

~ 18 minutes). Final masked reconstruction error: train 2.7×10^{-3} , validation 2.1×10^{-3} , with no sign of overfitting. Table 2 reports the measured linear-probe R^2 .

Table 2: Run 1: measured linear-probe R^2 on the 600-sample test split ($\lambda_{\text{decorr}} = 0.01$). Bold marks the block that should carry each target.

Target	z_μ	z_g	z_ξ
log Re	0.994	0.900	0.777
Area fraction	0.042	0.688	0.172
Centroid x	0.027	0.736	0.373
Centroid y	-0.013	0.690	0.214

What worked. The regime block is clean: z_μ reads log Re at $R^2 = 0.994$ and contains essentially no geometry ($R^2 \leq 0.04$). Geometry is readable from z_g ($R^2 \approx 0.69$ – 0.74) and nearly absent from z_μ . The moderate (rather than high) geometry recovery is not alarming: the probes are linear, while area and centroid are nonlinear functionals of the SDF that z_g encodes.

What failed. Reynolds-number information leaked into the other blocks: log Re is readable from z_g at $R^2 = 0.900$ and from z_ξ at 0.777. This is exactly the factor-scrambling that the impossibility result of Locatello et al. predicts for unconstrained latents: the decoder needs Re to reconstruct the field and is free to stash copies of it in every block, and the reconstruction loss rewards doing so. The decorrelation penalty is meant to prevent this, but at $\lambda_{\text{decorr}} = 0.01$ it is roughly two orders of magnitude weaker than the reconstruction term and loses the competition.

Next experiment. Since the leaked Re is *linearly* recoverable from z_g — which is precisely what a Pearson penalty can act on — the next run raises λ_{decorr} from 0.01 to 0.5 with all else fixed. Success criteria: the off-diagonal log Re entries drop substantially (toward $R^2 \lesssim 0.3$) while the reconstruction error and the z_g geometry row remain near their Run-1 values. If a strong Pearson weight cannot remove the leakage, the natural escalation is HSIC (L7, nonlinear dependence) and the swap-consistency loss (L12), which makes the factorization functional rather than merely statistical.

6.2 Run 2: strong Pearson decorrelation

Identical to Run 1 except $\lambda_{\text{decorr}} = 0.5$ (a $50\times$ increase). Table 3 reports the measured linear-probe R^2 , with the Run-1 values for comparison.

Table 3: Run 2: measured linear-probe R^2 on the test split ($\lambda_{\text{decorr}} = 0.5$), with Run-1 values ($\lambda_{\text{decorr}} = 0.01$) in parentheses. Bold marks the block that should carry each target.

Target	z_μ	z_g	z_ξ
log Re	0.886 (0.994)	0.719 (0.900)	0.487 (0.777)
Area fraction	-0.002 (0.042)	0.402 (0.688)	0.262 (0.172)
Centroid x	-0.016 (0.027)	0.565 (0.736)	0.378 (0.373)
Centroid y	0.016 (-0.013)	0.427 (0.690)	0.115 (0.214)

Outcome: a Pareto trade-off, not a fix. The leakage shrank — log Re from z_g dropped from 0.900 to 0.719 and from z_ξ from 0.777 to 0.487 — but so did everything else: the on-diagonal regime readout degraded ($0.994 \rightarrow 0.886$) and the geometry recovery from z_g fell by roughly 0.2 across all three descriptors. The strong Pearson penalty cannot distinguish redundant copies of information (which we want removed) from informative, structured coordinates (which we want kept), so at large weight it partially scrambles all blocks while still leaving the leakage far above the $R^2 \lesssim 0.3$ target. A $50\times$ increase in decorrelation pressure

bought a modest leakage reduction at a real cost — strong evidence that weight tuning of a statistical independence penalty alone will not functionally separate the blocks.

Next experiments. Two directions, in order: (i) $\lambda_{\text{decorr}} = 0.1$ as an intermediate sweep point — together with Runs 1–2 this begins the (reconstruction, identifiability) Pareto front that the project plan targets; (ii) *same-factor invariance* (L10): the dataset contains each geometry at multiple Reynolds numbers, so we can penalize the variance of $\mathbf{z}_g$ across samples that share a geometry. This demands directly that $\mathbf{z}_g$ be unchanged when only Re changes — a functional constraint on the factorization rather than a statistical one, and the first step toward the group-structured supervision at the heart of the framework’s identifiability argument.

7 Next steps

Deliberately excluded from this first version and planned as increments once the baseline trains and the probe table looks sane:

- swap-consistency loss (L12): decode $\mathbf{z}_\mu$ from one sample with $\mathbf{z}_g$ from another and compare against the true cross-combination;
- group-structured minibatches for iVAE-style identifiability;
- concept-vector arithmetic diagnostic;
- INR-style implicit decoder; HSIC (nonlinear) decorrelation;
- dynamics block $\mathbf{z}_\eta$ and latent propagator Φ for time-dependent cases.

A Appendix: Running the code on Nova cluster

- STEP1: Login into the Nova cluster `ssh username@nova.its.iastate.edu`
- STEP2: Activate the virtual environment that contains all the required packages.
- STEP3: Request resource allocation from Nova using `salloc -N 1 -n 4 -t 06:00:00 --gres=gpu:a100 --mem 369G`